

Project Demonstration Guide

Purpose

Demonstrate the complete role-based workflow in 10–15 minutes while stating accurately that checkout and donation payments are simulations.

Preparation

1. Start MongoDB.
2. Configure `server/.env` with `MONGODB_URI`, `PORT=5000`, and a local `JWT_SECRET`.
3. Run the server, then the Vite client.
4. Verify `GET http://localhost:5000/health`.
5. Confirm the home page shows the three seed temples.
6. Ensure future slots exist; seed slots are relative to the original initialization date and may need replacement later.
7. Prepare one account for each role and a normal user with known credentials.

The current application can create privileged roles from the public registration form, but this is a critical defect. Use that only to demonstrate the reviewed code locally; explain that production provisioning must be admin-controlled.

Seed content

Temple	Location	Typical seeded hours
Tirumala Venkateswara Temple	Tirupati, Andhra Pradesh	06:00 AM–09:00 PM
Kashi Vishwanath Temple	Varanasi, Uttar Pradesh	04:00 AM–11:00 PM
Ayodhya Ram Mandir	Ayodhya, Uttar Pradesh	06:00 AM–10:00 PM

Initial empty-database seeding creates four slots per temple across the next two days, with capacities 30–60 and prices INR 150 or 200.

Demo script

1. Guest discovery (2 minutes)

- Show the home hero and responsive temple cards.
- Search for `Kashi` and filter by `Varanasi`.
- Open the temple detail page.
- Point out hours, amenities, upcoming events, ratings, date selector, seat counts, audio, and light/dark theme.

2. Devotee booking (4 minutes)

- Sign in as a USER.
- Choose a slot and enter two devotee records.

- Create the hold and point out the five-minute countdown and immediate seat deduction.
- Enter syntactically valid demo card fields and confirm.
- Show the generated **DE**- ticket code, devotee details, amount, and printable ticket.
- Open the user dashboard and show booking history/profile.

Say explicitly: "The project simulates card validation. The backend receives no card details and performs no bank transaction."

3. Donation and review (2 minutes)

- Submit a temple review and show the updated average/list.
- Open Donations, select a purpose and amount, choose named/anonymous mode, and complete the simulated form.
- Show the **TXN**- receipt and donation history.

4. Organizer operations (2 minutes)

- Open the organizer dashboard.
- Create a future slot and show it in the list/temple page.
- Show the booking registry.
- Mention that notifications are currently hard-coded demo items.

5. Administrator operations (3 minutes)

- Show summary analytics and explain that "revenue" is recorded demo value, not settled funds.
- Filter users by role.
- Create a temple or schedule an event.
- Log a maintenance request and change its status.

Expected API evidence

Action	Endpoint	Expected status
Search temples	GET /api/temples	200
Create hold	POST /api/bookings	201
Confirm	POST /api/bookings/:id/confirm-payment	200
Donation	POST /api/donations	201
Create slot	POST /api/slots	201
Analytics	GET /api/admin/analytics	200

Questions and accurate answers

How is overbooking prevented?

The current controller checks available seats and deducts them before creating the booking. That works sequentially but is not safe under concurrent requests; an atomic conditional update/transaction is required.

How are expired holds released?

Selected booking and slot endpoints run a sweep that marks overdue pending bookings expired and returns seats. There is no background scheduler yet.

Is payment integrated?

No. Card-format validation and payment confirmation are simulations.

How is authorization handled?

JWT middleware loads the user and role middleware protects privileged APIs. Public role selection during registration is a critical defect documented for correction.

Are automated tests included?

No. The frontend build and backend syntax pass, but unit, integration, E2E, and concurrency tests must be added.

Is the application production-ready?

No. It is suitable for demonstration after local setup. The security, consistency, test, and operational blockers are documented in the testing phase.

Demo fallback

- If the database is offline, show `/health` and explain that `/api` correctly returns 503 while MongoDB is unavailable.
- If slots are in the past, create a future slot as organizer.
- If audio autoplay is blocked, interact with the page and use the player control.
- If a hold expires, reopen the modal and create a new hold.
- Do not claim a real payment, refund, message notification, or production deployment.